EXECUTIVE SPECIFICATION & EXECUTION BLUEPRINT

TrustVault: Executive & Security Blueprint

A plain-English guide to building, defending, and acing the TrustVault challenge.

Targets: 100/100 Marks | Non-Negotiables | Attack Playbook | THREATS.md | PROMPTS.md

01 The Situation in Plain English

Every company shares highly confidential files: executive salary sheets, customer NDAs, trade secrets, and audit records. In standard software, two catastrophic failures happen constantly: (1) Broken permissions where someone is added to a folder for one file and inherits forty files, or where an ex-employee's link still works; and (2) Zero accountability, where a leak occurs and nobody can answer who opened the file or when.

TrustVault is a secure document platform built to solve exactly this: a platform where the wrong person cannot reach a document through ANY route, and where you can mathematically prove exactly who reached it, including failed attempts.

CRITICAL ASSESSMENT REALITY: THIS IS NOT A STANDARD CRUD APP

The challenge author explicitly states: 'This is a security problem wearing the clothes of a CRUD application.' Many candidates spend 80% of their time making a fancy frontend with drag-and-drop animations, but fail the assessment. If a judge can run 'curl https://your-app/api/docs/42' with an unauthorized user's token and get a 200 OK, you lose almost all 35 Security marks immediately. The backend authorization engine is what is being judged.

02 The 3 Non-Negotiable Rules (Fail Any = Fail System)

The specification defines three non-negotiable rules. If your system violates any of these, it fails:

- Rule 1: Server-Side Decides Every Time. Never rely on hiding a button, disabling a menu, or client-side routing. Every single API endpoint, file stream, and search query must independently verify authorization against the database before returning a single byte.
- Rule 2: Provable Audit Trail (Successes AND Denials). You must be able to prove who reached a file, when, and from what IP. Crucially, you must also record REFUSED ATTEMPTS (HTTP 403). If Alice tries to access Bob's salary sheet and is blocked, that denied attempt must be logged.
- Rule 3: Instant Revocation (Zero Window of Vulnerability). The millisecond an owner revokes access from a user, any links, active sessions, or pending requests for that document must cease functioning immediately. No 15-minute token caching delays.

03 The 10 Required Features (The Baseline Floor)

The specification lists 10 baseline requirements. All ten must be cleanly implemented:

#	Feature	Specification	Key Security Implementation
1	Authentication	At least 3 roles: Admin, Owner, Viewer	Bcrypt password hashing, strict session/token au
2	Document Management	Upload, organise, search, view, delete	Files stored outside public root; UUID file name
3	Granular Permissions	Read vs Edit granted to named users	Normalized permissions table; per-document grant
4	Owner Delegation	Grant access and take it back	Only Document Owner (or Admin) can alter rights
5	Strict Access Wall	Unauthorized users blocked from all ops	Server-level 403 Forbidden on read/download/edit
6	Tamper-Evident Audit	Log upload, download, edit, delete, perms	Append-only table; includes refused/denied attem
7	Multi-Param Search	Search by name, owner, category, date	SQL WHERE clause restricted to accessible docs o
8	Upload Validation	Validate size, MIME type, extensions	Sniff magic bytes (%PDF, PK); sanitize file name
9	Admin Oversight	Activity view, permission changes, alerts	Admin sees audit trail & access graphs; not file
10	Automated Tests	Automated proof of cross-user isolation	Integration tests simulating unauthorized curl a

04 The 100-Point Marking Rubric Breakdown

Your grade is determined across six distinct bands:

Marks	Category	What the Judges Look For	How to Win
35 Marks	Security	Judge attacks your live deployed URL with automa	Close all IDOR, path traversal, delayed token bu
15 Marks	Threat Thinking	THREATS.md compared against the evaluators' mast	Be brutally honest about threats found, fixed, o
15 Marks	Functionality	The 10 baseline requirements functioning smoothl	Clean UI/API covering all operations end-to-end
15 Marks	AI Usage	PROMPTS.md showing 5+ dated entries catching AI	Document where AI suggested flawed security and
10 Marks	Engineering Quality	Clean architecture, error handling, defensive pr	Modular structure, separation of concerns, no me
10 Marks	Documentation & Demo	Clean README, seed accounts, clean clone deploy,	Single-command docker/setup script; clear video

THE 'ONE DOCUMENT' RULE (DON'T LOSE 35 MARKS)

The spec notes: 'One document reached by someone who should not have reached it costs most of the first band, whatever else works.'

This means if your app has 99 features working, but an unauthorized user can fetch document #2 by modifying an API ID, you lose 35 marks immediately.

Security is not a feature; it is the entire evaluation.

05 The Judges' Attack Playbook (7 Exploits & Defenses)

A judge will spend dedicated time attempting to bypass your permissions. Below are the exact attacks they will execute and the architecture required to stop them:

Attack 1: BOLA / IDOR (Insecure Direct Object Reference)

- The Attack: User Alice is granted Doc 1. Alice inspects network traffic, sees 'GET /api/documents/1/download', and sends 'GET /api/documents/2/download' (Bob's salary letter).
- The Flaw: Code queries 'SELECT * FROM documents WHERE id = :id' without checking if Alice owns it or has an active grant.
- The Fix: Run an authorization query joining 'DocumentPermissions' and 'current_user_id'. Return HTTP 403 immediately and log the denied attempt.

Attack 2: Direct Static Storage Bypass (URL Guessing)

- The Attack: Judge guesses the storage path: 'https://your-domain.com/uploads/salary_bob.pdf' or discovers the cloud S3 bucket URL.
- The Flaw: Saving uploaded files directly into a public static directory (e.g. 'public/' or 'static/uploads/').
- The Fix: Store files outside web root (e.g. 'storage/vault/' with UUID names like 'a7f9-4b12...bin'). Serve files ONLY through an authenticated streaming controller.

Attack 3: Delayed Revocation Window (Token Caching)

- The Attack: User Charlie is granted access. Owner revokes Charlie's access. Charlie immediately hits download using his existing session/token.
- The Flaw: Relying on stateless JWT tokens containing 'permissions: ['read_doc_1']' with 1-hour expiry.
- The Fix: Never embed document permissions inside JWTs! Always query live permission records in the database on every download request.

Attack 4: Privilege Escalation on Permissions API

- The Attack: A Viewer sends 'POST /api/documents/1/permissions' with '{'user_id': 'charlie', 'permission': 'edit'}'.
- The Flaw: Endpoint checks if user is logged in, but fails to check if 'current_user_id == document.owner_id'.
- The Fix: Enforce role/owner validation on every grant/revoke endpoint.

Attack 5: Malicious File Upload (XSS & Path Traversal)

- The Attack: Judge uploads a file named ‘../../etc/passwd‘ or an SVG file containing ‘<script>alert(document.cookie)</script>‘.
- The Flaw: Trusting user-supplied filename and using it in file paths; or serving SVGs with ‘Content-Type: image/svg+xml‘ without CSP headers.
- The Fix: Strip filenames, store as UUIDs on disk, sniff file magic bytes (verify PDF starts with ‘%PDF-‘), and serve downloads with ‘Content-Disposition: attachment; filename="safe.pdf"‘.

Attack 6: Metadata Leakage via Search & Error Messages

- The Attack: Unauthorized user searches ’Confidential Acquisition’ or types random IDs to observe differences in error responses (404 vs 403).
- The Flaw: Global document search query without permission filtering, or leaking document titles in error responses.
- The Fix: Search SQL query must include ‘JOIN document_permissions‘ to only match docs the caller has rights to view.

Attack 7: Audit Log Tampering (Covering Tracks)

- The Attack: Attacker attempts to delete or update rows in ‘audit_logs‘ table via SQL injection or admin UI.
- The Flaw: Providing an admin endpoint to ’Clear Logs’ or editable audit records.
- The Fix: Audit log table is strictly append-only. No UPDATE or DELETE SQL statements exist anywhere in the codebase.

06 The Core Architecture Pattern

Below is the bulletproof authorization and streaming pattern for every document request:

Python/Flask: Secure Streaming & Audit Enforcement Pattern

```
@app.route(’/api/documents/<doc_id>/download’, methods=[’GET’])
@login_required
def download_document(doc_id):
    user = current_user
    doc = Document.query.get_or_404(doc_id)

    # THE CRITICAL LINE (Evaluated on every request):
    has_access = check_permission(user.id, doc.id, required=’read’)

    if not has_access:
        # RULE 2: Log refused attempt
        log_audit(user_id=user.id, doc_id=doc.id, action=’DOWNLOAD’, status=’DENIED’)
        abort(403, ’Access denied’)

    # Log successful access
    log_audit(user_id=user.id, doc_id=doc.id, action=’DOWNLOAD’, status=’ALLOWED’)

    # Safe isolated file streaming from non-public storage directory
    safe_path = os.path.join(VAULT_STORAGE_DIR, f’{doc.storage_uuid}.dat’)
    return send_file(safe_path, as_attachment=True, download_name=doc.original_filename)
```

07 How to Write THREATS.md (15 Marks)

THREATS.md is the document that separates winning submissions from average ones. The evaluators maintain their own secret list of threats and match yours against it. Marks are awarded for threats you found and closed, extra marks for threats they didn't think of, and marks for HONESTLY accepting risks.

THE GOLDEN RULE OF THREATS.MD: HONESTY SCORES HIGH

An entry saying: 'I know this system is vulnerable to memory-exhaustion DoS if a user uploads 100 simultaneous 25MB files, and I accepted this risk because implementing a Redis distributed queue was out of scope for a 7-day sprint' IS WORTH FULL MARKS.

Pretending a hole doesn't exist scores zero because the judges will find it.

Do not stop at 4 obvious entries. Provide 8 to 12 rigorous threat analyses.

Required Structure for Each Entry in THREATS.md

- 1. Threat Title & Category: e.g., IDOR on Document Download API (OWASP API1:2023 Broken Object Level Authorization).
- 2. Attack Vector: How an attacker would exploit it (exact HTTP method, route, parameters, and headers).
- 3. Mitigation Implemented: Specific defensive code (e.g. SQL join on permission table, server-side UUID storage).
- 4. Automated Test Verification: Name of test case in test suite proving the attack is blocked (e.g., 'test_unauthorized_download_denied').
- 5. Residual / Accepted Risk: What remains open and why (e.g., timing attack on doc existence vs 404 response).

08 How to Write PROMPTS.md (15 Marks)

PROMPTS.md is your AI interaction record. Writing it on the last day scores zero. It must show evidence of directing AI tools and catching them being confidently wrong.

PROMPTS.MD CRITICAL REQUIREMENTS (DON'T MISS THESE)

1. At least 5 dated entries spread across the week.
2. At least TWO entries must be specifically about Authorization, Access Control, or Audit Trail.
3. Each entry must follow the 4-part structure: (a) What you asked; (b) What AI produced; (c) What was WRONG with it specifically; (d) What you did instead, with a link to the Git commit where you fixed it.
4. 'It was all fine' or 'AI was helpful' scores ZERO marks. You must show the security flaw the AI generated and how you caught it.

Sample High-Scoring Entry for PROMPTS.md

- Date & Commit: Nov 4, 2026 | Commit ‘a4f89c1‘ (fix: enforce server-side auth on download route)
- What I Asked: ’Generate a Flask endpoint to download documents given a document_id.’
- What the AI Produced: ‘send_file(f’uploads/{doc_id}.pdf’)‘ with only ‘@login_required‘ checking if the user was logged in.
- What was Wrong (Security Flaw): Massive BOLA / IDOR vulnerability! The AI verified authentication (logged in), but failed authorization (did the user have access to THIS specific document?). Any logged-in user could download confidential salary docs of other users.
- What I Did Instead: Rewrote the endpoint to query ‘DocumentPermission‘ joining ‘current_user.id‘. If no row exists, immediately recorded an audit log entry with ‘status=’DENIED’‘ and returned HTTP 403 Forbidden. Added unit test ‘test_cross_tenant_download_blocked‘ in ‘tests/test_security.py‘.

09 The Live Interview Preparation: The ’One Question’

Shortlisted candidates undergo a live code walkthrough where the interviewer will ask you to make a live change. Expect the following exact question:

> **THE INTERVIEW TRAP QUESTION**
> Interviewer Question: ’Show me where the authorization decision is made when someone downloads a file. What would happen if that line were removed?’
> What they are testing:
> 1. Do you know your own code? (Can you jump directly to the exact controller and line without hesitation?)
> 2. The Tear-down Test: If that authorization line is removed and the test suite is executed, DO THE TESTS FAIL? If tests still pass, your tests were fake and test nothing.

Your Model Answer in the Interview:

1. Open ‘services/auth_service.py‘ at function ‘can_access_document(user_id, doc_id, ’read’)‘. 2. Explain: ’If this line is removed, the function falls through and unauthorized users receive the file stream. However, our test suite in ‘tests/test_auth_isolation.py‘ runs ‘test_unauthorized_download_fails()‘. If you comment out this check, the test immediately fails with an assertion error expecting 403 but getting 200.’ 3. Live Demo: Comment out the line in front of them and run ‘pytest‘. Watch the red test failure prove that your security net is airtight.

10 Recommended Database Schema & Entity Design

A clean, normalized schema is the foundation of provable security. Below is the recommended design:

Table	Key Columns	Security Purpose
users	id, email, password_hash, role (admin/owner/view	Stores credentials with bcrypt; baseline identit
documents	id, owner_id, original_name, storage_uuid, size,	Metadata record; file stored safely on disk by s
document_permissions	id, document_id, user_id, permission (read/edit)	Explicit grants to named users; real-time revoca
audit_logs	id, user_id, document_id, action, status (ALLOWE	Append-only security log; proves who reached or

11 7-Day Day-by-Day Execution Roadmap

Follow this systematic timeline to guarantee completion without last-minute panic:

Day	Milestone Focus	Deliverables Produced	Git / AI Commit Goal
Day 1	Foundation & Auth	DB schema, Flask/FastAPI setup, Bcrypt auth, 3 r	Commit seed data & schema; log prompt #1
Day 2	File Storage & Stream	MIME sniffing upload, UUID storage, safe stream	Validate magic bytes; block static folder bypass
Day 3	Permission Engine	Grant/Revoke API, instant revocation, zero cachi	Log prompt #2 on auth flaw caught; write tests
Day 4	Audit & Denied Logging	Audit log model, log 403 denied attempts, admin	Log prompt #3 on audit tamper protection
Day 5	Security Test Suite	Automated tests for IDOR, path traversal, revoca	Automated tests proving unauthorized access bloc
Day 6	THREATS.md & Cloud Deploy	Deploy to Render/Railway early; verify live URL	Log prompt #4 & #5; commit comprehensive THREATS
Day 7	Video Demo & Write-Up	5-minute screen recording, final README polish,	Final submission tag; clean clone verification

REMINDER: DEPLOY EARLY IN THE WEEK

The challenge instructions explicitly warn: 'Deploy early in the week, not on the last day. People who leave it to the end usually do not deploy at all.'

Deploy your database and backend to Render, Railway, or Fly.io by Day 5 or 6 so you have full confidence in your live URL.

12 The Final Submission Checklist

Before handing in your project, verify every item on this checklist:

- [ ] Public Git Repository: Has commits spread regularly across the entire week (no single massive commit at the end).
- [ ] Live Working URL: Accessible online with seed data loaded and tested from an incognito browser.
- [ ] README.md with Credentials: Contains login credentials for at least one Admin, one Document Owner, and one Viewer.
- [ ] THREATS.md Completed: Lists 8-12 security threats analyzed, how each was mitigated, and accepted risks with technical justifications.
- [ ] PROMPTS.md Completed: Contains at least 5 dated entries following the 4-part structure, with at least 2 entries on Authorization/Audit.
- [ ] Automated Security Tests: Running ‘pytest‘ passes 100% of tests. Removing an authorization check makes tests fail.
- [ ] Database Schema & Setup Scripts: A clean clone can run a single command (e.g. ‘docker compose up‘ or ‘python init_db.py‘) to initialize.
- [ ] API Documentation: Clearly documents each endpoint and specifies exactly which role/permission is required to call it.
- [ ] 5-Minute Demo Video: Walks through: (1) Roles; (2) Upload & permission grant; (3) Unauthorized access refusal; (4) Instant revocation; (5) Audit log inspection.
- [ ] Short Architectural Write-Up: Summary of what you built, what you deliberately did not build, and your permission model in your own words.

FINAL WORD OF ENCOURAGEMENT

TrustVault is an extraordinary project to showcase true security engineering skill.
By focusing on server-side authorization, tamper-resistant audit logs of denied attempts, and instant revocation, you will stand far above candidates who only build visual UI screens.
Keep your code simple, your authorization explicit, and your tests rigorous. You’ve got this!